

CS101: Introduction to Algorithms

Course overview: asymptotic analysis, sorting, searching, graphs, and dynamic programming.

Instructor: Dr. Ada Lovelace. Term: Fall.

Learning goal: reason about correctness and runtime of classic algorithms.

Big-O Notation

Definition

$f(n) = O(g(n))$ if there exist constants $c > 0$ and n_0 such that for all $n \geq n_0$, $0 \leq f(n) \leq c * g(n)$.

- Big-O gives an asymptotic upper bound on growth.
- Ignore constant factors and lower-order terms.
- Common classes: $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$, $O(2^n)$.

Theta and Omega

Omega (Ω) — lower bound. Theta (Θ) — tight bound.

- Example: $3n + 2 = \Theta(n)$
- $n^2 + n = O(n^2)$ and also $\Omega(n^2)$.

Loop Analysis Warmup

A single loop from 1 to n doing constant work is $\Theta(n)$

Nested loops $i=1..n$, $j=1..n$ doing constant work is $\Theta(n^2)$

```
for i in 1..n:  
  for j in 1..n:  
    work()
```

Binary Search

Binary search finds a target in a sorted array by repeatedly halving the search space.

- Requires the array to be sorted.
- Time complexity: $O(\log n)$ comparisons.
- Space complexity: $O(1)$ iterative, $O(\log n)$ recursive stack.

Binary Search Pseudocode

```
lo=0, hi=n-1
while lo <= hi:
    mid = (lo+hi)//2
    if A[mid]==target: return mid
    if A[mid]<target: lo=mid+1
    else: hi=mid-1
return NOT_FOUND
```

Linear Search Contrast

Linear search scans each element until a match or end of array.

- Works on unsorted data.
- Worst-case time: $\Theta(n)$
- Prefer binary search when data is sorted and queries are frequent.

Sorting Landscape

Comparison sorts require $\Omega(n \log n)$ comparisons in the worst case (decision-tree lower bound).

- Insertion sort: simple, good for nearly sorted arrays.
- Merge sort: guaranteed $O(n \log n)$, needs extra memory.
- Quicksort: average $O(n \log n)$, worst $O(n^2)$ without care.

Insertion Sort

Builds a sorted prefix by inserting each next element into its place.

- Best case (already sorted): $\Theta(n)$
- Worst case (reverse sorted): $\Theta(n^2)$
- Stable and in-place.

Merge Sort

Divide array in half, recursively sort, then merge two sorted halves.

- Recurrence: $T(n) = 2T(n/2) + \Theta(n)$ $\Rightarrow T(n) = \Theta(n \log n)$
- Requires $\Theta(n)$ extra space for merging.
- Stable sort.

Merge Procedure

Two pointers walk left and right halves; always take the smaller head into the output.

```
while i < |L| and j < |R|:  
  if L[i] <= R[j]: out <- L[i++]  
  else: out <- R[j++]
```

Quicksort Partition

Choose a pivot; rearrange so smaller elements are left, larger right.

- Lomuto partition is common in teaching.
- Randomized pivot selection avoids adversarial worst case with high probability.

Quicksort Complexity

Average case $\Theta(n \log n)$ Worst case $\Theta(n^2)$ Partitions are always unbalanced.

- In-place (aside from recursion stack).
- Not stable in typical implementations.

Heap Sort Snapshot

Build a max-heap, then repeatedly extract-max into the sorted suffix.

- Time: $\Theta(n \log n)$ worst case.
- In-place; not stable.

Hash Tables Intro

Hash tables map keys to buckets via a hash function for expected $O(1)$ lookup.

- Collisions handled by chaining or open addressing.
- Load factor ; $\hat{O} \approx \hat{O} - \text{æfÇVVæ6W2} \text{ W\&`ormance.}$

Graphs: Definitions

A graph $G = (V, E)$ has vertices V and edges E .

- Directed vs undirected; weighted vs unweighted.
- Degree of a vertex is the number of incident edges.
- Path: sequence of vertices connected by edges.

Graph Representations

Adjacency matrix: $|V| \times |V|$ matrix; $O(1)$ edge query; $\Theta(|V|^2)$ space.

Adjacency list: array of neighbor lists; $\Theta(|V| + |E|)$ space; better for sparse graphs.

BFS

Breadth-First Search explores level by level using a queue.

- Finds shortest paths in unweighted graphs.
- Time: $O(V + E)$

DFS

Depth-First Search explores as deep as possible using a stack (or recursion).

- Useful for topological sort, cycle detection, connected components.
- Time: $O(V + E)$

Dijkstra Overview

Dijkstra finds shortest paths from a source in graphs with non-negative edge weights.

- Uses a priority queue of tentative distances.
- With binary heap: $O((|V| + |E|) \log |V|)$.
- Does NOT handle negative edge weights correctly.

Dynamic Programming Idea

DP solves overlapping subproblems with optimal substructure by storing sub-results.

- Memoization (top-down) or tabulation (bottom-up).
- Classic examples: Fibonacci, knapsack, LCS, edit distance.

0/1 Knapsack

Given weights w_i , values v_i , and capacity W , maximize value without exceeding W ; each item at most once.

- DP table $dp[i][w]$ = best value using first i items with capacity w .
- Time: $O(nW)$.

Fibonacci DP

Naive recursion is exponential due to repeated subproblems.

```
F[0]=0; F[1]=1  
for i=2..n: F[i]=F[i-1]+F[i-2]  
return F[n]
```

- Linear time, constant extra space with two rolling variables.

Amortized Analysis Note

Amortized analysis averages cost over a sequence of operations.

- Dynamic array doubling: append is amortized $O(1)$.
- Aggregate, accounting, and potential methods are common techniques.

P vs NP (Survey)

P: problems solvable in polynomial time. NP: verifiable in polynomial time.

- NP-complete problems are the hardest in NP under poly-time reductions.
- Examples often cited: SAT, traveling salesperson decision version.

This course only surveys the idea; proofs are out of scope.

Exam Strategy

Show recurrence + closed form or Master Theorem when analyzing divide-and-conquer.

- State assumptions (sorted input, non-negative weights).
- Cite invariants for loop correctness sketches.

Summary Slide

You should know: Big-O/Theta/Omega, binary search, insertion/merge/quick sort, BFS/DFS, Dijkstra limits, basic DP.

Next: practice problems in the study notes and check the syllabus for grading weights.

Practice: Big-O Ranking

Rank for large n : $\log n < \sqrt{n} < n < n \log n < n^2 < 2^n < n!$.

- Polynomials always lose to exponentials eventually.
- Stirling approximates $n! \sim \sqrt{2\pi n} \left(\frac{n}{e}\right)^n$

Practice: Loop Nests

Outer $1..n$, inner $1..i$ → about $n(n+1)/2$ iterations → $\Theta(n^2)$
 Halving loops (while $n = n/2$) contribute $\Theta(\log n)$ factors.

Counting Sort Mention

Non-comparison sort for integers in a known range $0..k$.

- Time $\Theta(k + n)$ not comparison-based, so can beat $n \log n$.
- Out of deep scope; know it exists.

Stable Partition Idea

A stable partition preserves relative order of equal keys.

Useful prelude to understanding why merge sort stays stable.

BFS Layering Example

Start at s ; layer 0 = $\{s\}$; layer d = unvisited neighbors of layer $d-1$.

- First time a node is reached is a shortest unweighted path.

DFS Edge Types

Tree edges form the DFS forest. Back edges signal cycles in directed graphs.

- Forward/cross edges appear depending on discovery order.

Priority Queues for Dijkstra

Binary heap: decrease-key or re-insert patterns both taught.

- Fibonacci heaps improve asymptotics but are rarely implemented in class.

Bellman-Ford Contrast

Relax all edges $|V|-1$ times; detects negative cycles with one more pass.

- Slower than Dijkstra but handles negative weights.
- CS101 only surveys this algorithm.

LCS Recurrence

If $X[i]=Y[j]$, $LCS(i,j)=1+LCS(i-1,j-1)$; else $\max(LCS(i-1,j), LCS(i,j-1))$.

- Table size $\Theta(mn)$ for strings of length m,n .

Edit Distance Sketch

Levenshtein distance: insert, delete, substitute costs usually 1.

- Same DP shape as LCS-family problems.

Array Doubling Potential

When capacity doubles on overflow, costly resizes are rare.

- Amortized append cost $O(1)$ via aggregate or potential :b Ò &â Ò 6 6—G•.

Reduction Intuition

A poly-time reduction $A \leq_p B$ means B is at least as hard as A (up to poly factors).
Used to define NP-completeness; proofs out of scope.

Invariant Example

Binary search invariant: target in $A[\text{lo}..\text{hi}]$ if present anywhere initially.
State the invariant before the loop and argue it is preserved.

Final Checklist Slide

- Definitions: O , Θ , $\sim$.
- Search & sort families with stability/space.
- BFS/DFS/Dijkstra assumptions.
- One DP table filled by hand.